Continuous Security Threat-Model Maintenance for Tool-Using Agents

Keeping assumptions, conclusions, controls, and evidence current as agent systems evolve

Agent Security | Threat Modeling | Runtime Assurance | Security Testing

Parsa Oryani | Sharif University of Technology | Research interests: secure and trustworthy agentic AI

Research thesis

Tool-using agents change after deployment: tools are replaced, permissions are delegated or revoked, memory persists, models and prompts are updated, and external services alter behavior. A threat model created for an earlier configuration can silently retain conclusions whose assumptions or validation evidence are no longer current.

This proposal studies a runtime dependency model that links observable agent state to threat-model assumptions, threats, conclusions, controls, and tests. After a change, the system should identify the conclusions that require reassessment, preserve those that remain supported, and mark conclusions as unknown when the required evidence is missing or stale.

Problem and scope

The first version focuses on changes to tools, effective permissions, delegated authority, memory access, enforcement configuration, and model or prompt version. A change event is accepted only with a source, timestamp, artifact identity, and state version. The method does not infer safety from a trace-derived architecture diagram alone and does not assume that every relevant runtime change is observable.

A running example is a support agent that reads confidential records and sends notifications. Registering a new external destination is different from granting the agent authority to use it; changing an unrelated label should not invalidate the disclosure conclusion; and losing visibility into effective credentials should change the conclusion to unknown rather than safe.

Closest-work positioning

Run-time Threat Models already support continuous risk assessment through design-level and instance-level models.[1] Continuous threat-analysis pipelines and threat-model-driven testing also connect system evolution to reviewed threats and tests.[2][3] SAGA and ACE provide concrete agent security architectures whose policy, token, and execution state can serve as evidence.[4][5] The proposed distinction is a selective, evidence-aware maintenance procedure for agent-specific changes, not a claim to invent continuous threat modeling.

Research questions

RQ1. Which agent state properties and dependency relations are necessary to identify every reviewed conclusion and test affected by a change?

RQ2. Can selective reassessment match the relevant updates found by full reassessment while reducing unnecessary checks and reviewer effort?

RQ3. How should missing, delayed, conflicting, or stale observations change the status and explanation of a security conclusion?

RQ4. Which dependency patterns remain stable across agent frameworks, and which require framework-specific adapters or control evidence?

Method

1. Versioned security graph. Represent capabilities, tools, principals, permissions, data classes, memory stores, assumptions, threats, conclusions, controls, tests, and evidence as typed, versioned nodes. Edges record support, contradiction, mitigation, coverage, and dependence.

2. Trusted observations. Collect configuration and authorization evidence through bounded adapters. Each observation carries source identity, collection time, version, freshness policy, and confidence state. Unsupported or missing facts remain explicit unknowns.

3. Change impact semantics. Translate a state delta into affected graph predicates, propagate invalidation through declared dependencies, and produce one of three outcomes for each conclusion: supported, reassessment required, or unknown. New paths must be discovered from capability and authority changes rather than only from paths present in the original model.

4. Evidence-aware test selection. Link each conclusion to validating controls and tests. Select tests whose assumptions or covered paths changed, while keeping prior results as historical evidence rather than treating them as current proof.

5. Human-readable update receipt. For every change, report the observed event, affected assumptions, resulting conclusion status, selected tests, evidence age, and unresolved observation gaps. The explanation must distinguish a changed risk from a loss of knowledge.

Expected technical artifact

The prototype will include the typed graph schema, adapters for a small agent stack, an incremental update engine, a full-reassessment reference implementation, and a review interface that shows why each conclusion changed and which evidence would resolve an unknown state.

Evaluation plan

Evaluation uses small reproducible agents with controlled changes to tools, destinations, permissions, memory, policies, and model configuration. Each scenario contains an initial state, a change event, a resulting state, independently reviewed expected effects, and the evidence available to the algorithm. Cases include irrelevant changes, newly enabled paths, revoked authority, stale tests, missing observations, and conflicting data sources.

Selective maintenance is compared with full reassessment, naive invalidate-all, dependency reachability without evidence semantics, and no update. Primary measures are missed affected conclusions, unnecessary invalidations, stale-evidence detection, unknown-state accuracy, update latency, and reviewer effort. Case-level disagreements and adjudication are retained; a limited scenario set is reported as a case study rather than population-level assurance.

A portability study implements the same change classes in two agent frameworks or orchestration styles. It measures which graph concepts remain unchanged, how much adapter-specific logic is required, and whether identical high-level changes produce consistent conclusion updates.

Expected contributions and limits

Expected contributions are: (i) agent-specific dependency semantics connecting runtime capability and authority changes to threat conclusions and evidence; (ii) an update procedure that handles stale and missing observations explicitly; and (iii) an empirical account of freshness, traceability, and reassessment cost. The work will not claim that maintaining a threat model automatically prevents attacks or improves downstream security outcomes without a separate causal study.

The method cannot detect changes outside its observation boundary. Incorrect dependency links can preserve a stale conclusion or trigger unnecessary review. Human reference judgments may disagree, and some changes may require new threat discovery rather than incremental maintenance. The initial prototype excludes arbitrary multi-agent concurrency and enterprise-scale asset inventories.

Selected references

[1] S. Verreydt, D. Van Landuyt, and W. Joosen. Run-time Threat Models for Systematic and Continuous Risk Assessment. Software and Systems Modeling, 2025. doi:10.1007/s10270-024-01242-5.

[2] L. Sion et al. Automated Threat Analysis and Management in a Continuous Integration Pipeline. IEEE SecDev, 2021. doi:10.1109/SecDev51306.2021.00021.

[3] M. Raciti, G. Bella, and D. Van Landuyt. Threat Model-Driven Test Framework for Security and Privacy of Agentic LLM Applications. IFIP SEC, 2026. doi:10.1007/978-3-032-27993-4_6.

[4] G. Syros et al. SAGA: A Security Architecture for Governing AI Agentic Systems. NDSS, 2026. doi:10.14722/ndss.2026.230869.

[5] E. Li et al. ACE: A Security Architecture for LLM-Integrated App Systems. NDSS, 2026. doi:10.14722/ndss.2026.230352.